

Geometric Transformations Using OpenCV

Developed By : SRI HARI KRISHNA D T

Register No : 212224240160

Aim

To write a Python program using OpenCV to perform various geometric transformations on an image.

The program performs the following operations:

- Image Translation
- Image Scaling (Resizing)
- Image Shearing
- Image Reflection (Flipping)
- Image Rotation

Software Used

- Anaconda – Python 3.7
- Jupyter Notebook / VS Code
- OpenCV (cv2)
- NumPy
- Matplotlib

Algorithm

Step 1:

Import the required libraries: OpenCV, NumPy, and Matplotlib.

Step 2:

Read the input image in color mode.

Step 3: Image Translation

- Create a translation matrix to shift the image
- Move the image 50 pixels to the right and 80 pixels down
- Apply transformation using `cv2.warpAffine()`
- Display original and translated images

Step 4: Image Scaling

- Resize the image to 0.5× (downscale)
- Resize the image to 2× (upscale)
- Use `cv2.resize()`
- Display original, downscaled, and upscaled images

Step 5: Image Shearing

- Create transformation matrices for:
 - Horizontal shearing
 - Vertical shearing
- Apply transformations using `cv2.warpAffine()`
- Display original and sheared images

Step 6: Image Reflection

- Perform flipping using `cv2.flip()` :
 - Horizontal reflection
 - Vertical reflection
 - Both axes
- Display all reflected images

Step 7: Image Rotation

- Create rotation matrices for:
 - 45° rotation
 - 90° rotation
- Use `cv2.getRotationMatrix2D()` and `cv2.warpAffine()`
- Display original and rotated images

Program

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Step 1: Load the image
image = cv2.imread('Qn4.jpg') # Load the image from file

# Display the original image
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB)) # Convert BGR to RGB for correct display
plt.title("Original Image")
plt.axis('off')
print("SRI HARI KRISHNA D T")
print("REG NO: 212224240160")
```

```
# Step 2: Image Translation
tx, ty = 100, 50 # Translation factors (shift by 100 pixels horizontally and 50 vertically)
M_translation = np.float32([[1, 0, tx], [0, 1, ty]]) # Translation matrix:
# [1, 0, tx] - Horizontal shift by tx
# [0, 1, ty] - Vertical shift by ty
translated_image = cv2.warpAffine(image, M_translation, (image.shape[1], image.shape[0]))
```

```
plt.imshow(cv2.cvtColor(translated_image, cv2.COLOR_BGR2RGB)) # Display the translated image
plt.title("Translated Image")
plt.axis('off')
print("SRI HARI KRISHNA D T")
print("REG NO: 212224240160")
```

```
# Step 3: Image Scaling
fx, fy = 5.0, 2.0 # Scaling factors (1.5x scaling for both width and height)
scaled_image = cv2.resize(image, None, fx=fx, fy=fy, interpolation=cv2.INTER_LINEAR)
# resize: Resize the image by scaling factors fx, fy
# INTER_LINEAR: Uses bilinear interpolation for resizing
```

```
plt.imshow(cv2.cvtColor(scaled_image, cv2.COLOR_BGR2RGB)) # Display the scaled image
plt.title("Scaled Image") # Set title
plt.axis('off')
print("SRI HARI KRISHNA D T")
print("REG NO: 212224240160")
```

```
# Step 4: Image Shearing
shear_matrix = np.float32([[1, 0.5, 0], [0.5, 1, 0]]) # Shearing matrix
# The matrix shears the image by a factor of 0.5 in both x and y directions
# [1, 0.5, 0] - Shear along the x-axis (horizontal)
# [0.5, 1, 0] - Shear along the y-axis (vertical)
sheared_image = cv2.warpAffine(image, shear_matrix, (image.shape[1], image.shape[0]))
```

```
plt.imshow(cv2.cvtColor(sheared_image, cv2.COLOR_BGR2RGB)) # Display the sheared image
plt.title("Sheared Image") # Set title
plt.axis('off')
print("SRI HARI KRISHNA D T")
print("REG NO: 212224240160")
```

```
# Step 5: Image Reflection
reflected_image = cv2.flip(image, 2) # Flip the image horizontally (1 means horizontal flip)
# flip: 1 means horizontal flip, 0 would be vertical flip, -1 would flip both axes
```

```
plt.imshow(cv2.cvtColor(reflected_image, cv2.COLOR_BGR2RGB)) # Display the reflected image
plt.title("Reflected Image") # Set title
plt.axis('off')
```

```

print("SRI HARI KRISHNA D T")
print("REG NO: 212224240160")

# Step 6: Image Rotation
(height, width) = image.shape[:2] # Get the image height and width
angle = 45 # Rotation angle in degrees (rotate by 45 degrees)
center = (width // 2, height // 2) # Set the center of rotation to the image center
M_rotation = cv2.getRotationMatrix2D(center, angle, 1) # Get the rotation matrix
# getRotationMatrix2D: Takes the center of rotation, angle, and scale factor (1 means no
scaling)
rotated_image = cv2.warpAffine(image, M_rotation, (width, height)) # Apply rotation

plt.imshow(cv2.cvtColor(rotated_image, cv2.COLOR_BGR2RGB)) # Display the rotated image
plt.title("Rotated Image") # Set title
plt.axis('off')
print("SRI HARI KRISHNA D T")
print("REG NO: 212224240160")

# Step 7: Image Cropping
x, y, w, h = 100, 100, 200, 150 # Define the top-left corner (x, y) and the width (w) and
height (h) of the crop
# Cropping the image from coordinates (x, y) to (x+w, y+h)
cropped_image = image[y:y+h, x:x+w]
# The crop is performed by slicing the image array in the y and x directions

plt.imshow(cv2.cvtColor(cropped_image, cv2.COLOR_BGR2RGB)) # Display the cropped image
plt.title("Cropped Image") # Set title
plt.axis('off')
print("SRI HARI KRISHNA D T")
print("REG NO: 212224240160")

```

Output

Original Image

SRI HARI KRISHNA D T
REG NO: 212224240160

Original Image



Image Translation

SRI HARI KRISHNA D T
REG NO: 212224240160

Translated Image



Image Scaling

SRI HARI KRISHNA D T
REG NO: 212224240160

Scaled Image



Image Shearing

SRI HARI KRISHNA D T
REG NO: 212224240160

Sheared Image



Image Reflection

SRI HARI KRISHNA D T
REG NO: 212224240160

Reflected Image



Image Rotation

SRI HARI KRISHNA D T
REG NO: 212224240160

Rotated Image



Image Cropped

Cropped Image



Result

Thus, various geometric transformations such as translation, scaling, shearing, reflection, and rotation are successfully performed using OpenCV. These transformations demonstrate how images can be spatially manipulated for different computer vision applications.